```
# group 10
# nayan kv
# Chandy-Lamport Algorithm Simulation

class Process:
    def __init__(self, pid, local_state):
        self.pid = pid
        self.local_state = local_state
        self.neighbors = []            # Outgoing channels
        self.snapshot_taken = False
        self.channel_state = {}         # In-transit messages

    def add_neighbor(self, process):
        self.neighbors.append(process)

    def initiate_snapshot(self):
        print(f"\n🔴 Snapshot initiated at Process {self.pid}")
        self.record_local_state()
        self.send_marker()

    def record_local_state(self):
        self.snapshot_taken = True
        print(f"Process {self.pid} records local state: {self.local_state}")

        # Initialize channel state
        for neighbor in self.neighbors:
            self.channel_state[neighbor.pid] = []

    def send_marker(self):
        for neighbor in self.neighbors:
            print(f"Process {self.pid} sends MARKER to Process {neighbor.pid}")
```

```
            neighbor.receive_marker(self.pid)

    def receive_marker(self, from_pid):
        if not self.snapshot_taken:
            # First marker received
            self.record_local_state()
            self.send_marker()
        else:
            print(f"Process {self.pid} records channel state from Process {from_pid}")


P1 = Process(1, "Account Balance = 1000")
P2 = Process(2, "Account Balance = 2000")
P3 = Process(3, "Account Balance = 3000")

# Define communication channels
P1.add_neighbor(P2)
P2.add_neighbor(P3)
P3.add_neighbor(P1)

P1.initiate_snapshot()
```

OUTPUT

```
Snapshot initiated at Process 1
Process 1 records local state: Account Balance = 1000
Process 1 sends MARKER to Process 2
Process 2 records local state: Account Balance = 2000
Process 2 sends MARKER to Process 3
Process 3 records local state: Account Balance = 3000
Process 3 sends MARKER to Process 1
```

Process 1 records channel state from Process 3